UNIVERSITY OF TRIPOLI | FACULTY OF ENGINEERING | DEPARTMENT OF COMPUTER ENGINEERING

EXAMINATION COMMITTEE DEFENSE HANDOUT

Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Network

Candidate: Maher Abdulnasir Alqadhi (2210249576) **Supervisor:** Dr. Suad El-Geder
Degree: B.Sc. in Computer Engineering | EC499 Graduation Project **Academic Session:** Spring 2026 | Examination Committee Briefing

1 Executive Abstract & Research Rationale

Software-Defined Networking (SDN) centralizes control-plane intelligence but conventionally relies on static, load-agnostic routing protocols (such as Dijkstra SPF or ECMP). Under dynamic workloads, static routing causes **destination-based traffic pinning**, inducing severe core link saturation while parallel bypass capacity remains idle. This graduation project designs, implements, and benchmarks a closed-loop SDN traffic engineering platform integrating a **Dueling Double Deep Q-Network (D3QN)** agent, **Prioritized Experience Replay (PER)**, an empirical **4-tier candidate path diversity pipeline**, and an **Active/Standby 3-Controller Redundancy Pool** with OpenFlow 1.3 group table failover protection.

Key Empirical Breakthroughs (Verified Across 5 Fabrics & 120 Mininet Emulation Trials)

- **Bottleneck Utilization Relief:** Reduces grand mean bottleneck load from **63.88**% (Dijkstra SPF) to **30.88**% (a **51.7**% relative reduction; peak drop on Spine-Leaf: 99.87% → 20.89%, a **79.1**% drop).
- **End-to-End Latency Suppression:** Suppresses grand mean latency from **149.22** ms down to **44.54** ms (a **70.2**% latency cut).
- **High-Fidelity Wire Speed:** Achieves **64.89** Mbps throughput on a 65 Mbps physical bottleneck (**99.8**% link efficiency) with **0.45**% estimated packet loss.
- **Sub-Millisecond Control & Rapid Recovery:** Inference executed in **0.24** ms (> 4,000 decisions/s); local promotion in **0.001** ms and HTTP recovery in **37.96** ms.

2 Problem Formulation & Constrained Routing Objective

The traffic engineering problem is formalized as an unsplittable multi-commodity flow optimization on graph $G = (V, E)$. For active commodities $\mathcal{K}$, each demand d_k is assigned a single path $p \in \mathcal{P}_k$:

$$\min \max_{e \in E} \left(\frac{\sum_{k \in \mathcal{K}} \sum_{p \in \mathcal{P}_k : e \in p} d_k x_{k,p}}{C_e} \right) \quad \text{s.t.} \quad \sum_{p \in \mathcal{P}_k} x_{k,p} = 1, \quad x_{k,p} \in \{0, 1\}. \tag{1}$$

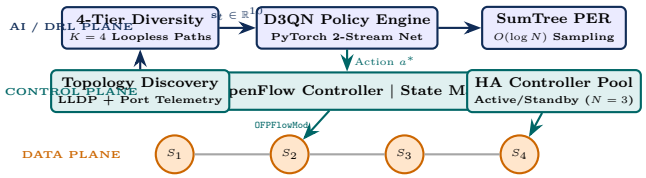


Figure 1: Closed-Loop SDN Architecture: AI Telemetry Ingestion, D3QN Inference, and OpenFlow Forwarding.

3 Mathematical Formulations & Algorithmic Design

3.1 1. Graph-Invariant 10-Dimensional State Vector (s_t)

To decouple inference from graph size, state vector $\mathbf{s}_t \in \mathbb{R}^{10}$ normalizes topological and traffic metrics into $[0.0, 1.0]$:

$$\mathbf{s}_t = \left[\hat{H}_{\min},\ \hat{u}_0,\ \hat{u}_1,\ \hat{u}_2,\ \hat{u}_3,\ \hat{D}_0,\ \hat{D}_1,\ \hat{D}_2,\ \hat{D}_3,\ \hat{u}_{\text{peak}}\right]^T, \tag{2}$$

where $\hat{H}_{\min} = \min(1, \text{hops}/10)$, $\hat{u}_i = \max_{e \in P_i} u_e$, $\hat{D}_i = \sum_{e \in P_i} d_e/50$ ms, and $\hat{u}_{\text{peak}} = \max_{e \in E} u_e$ is network-wide peak link utilization.

3.2 2. 4-Tier Hybrid Candidate Route Generation Pipeline

Action space $\mathcal{A}(s,d) = \{P_0, P_1, P_2, P_3\}$ is constrained to 4 loopless simple paths synthesized via `state_manager.py`:

- **Tier 0 (P_0):** Minimum-hop shortest path via Dijkstra's algorithm: $P_0 = \text{Dijkstra}(G, s, d)$.
- **Tier 1 (P_1):** Second shortest simple path synthesized via Yen's deviation spur search: $P_1 = \text{Yen}_2(G, s, d)$.
- **Tier 2 (P_2):** Diverse bypass path minimizing edge overlap with P_0: $P_2 = \arg\min_{P \in \mathcal{P}_{\text{simple}} \setminus \{P_0, P_1\}} |E(P) \cap E(P_0)|$.
- **Tier 3 (P_3):** Dynamic load-aware path via Dijkstra using nonlinear cost $c_{\text{dynamic}}(e) = d_e \cdot \left[1 + 25 \cdot (\max(0,\ u_{\text{eff}}(e) - 0.55))^2\right]$.

Instantaneous shadow reservation $u_{\text{shadow}}(e,t) = \sum_f \frac{R_f}{C_e} 2^{-(t-t_0)/\tau_{1/2}}$ ($\tau_{1/2} = 1.5$ s) damps transient micro-burst pileup.

3.3 3. Centered Dueling Double DQN & Implemented Quadratic Reward

Double DQN decouples action selection from target evaluation, updated via Polyak soft-updates $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ ($\tau = 0.005$):

$$Y_t^{\text{DDQN}} = R_{t+1} + \gamma Q\left(S_{t+1}, \arg\max_{a' \in \mathcal{A}} Q(S_{t+1}, a'; \theta);\ \theta^-\right), \quad \gamma = 0.95. \tag{3}$$

The Dueling architecture factors $Q(s,a)$ into scalar state value $V(s)$ and advantage $A(s,a)$ with mean-subtraction identifiability:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a' \in \mathcal{A}} A(s,a';\theta,\alpha)\right). \tag{4}$$

The routing engine optimizes the implemented quadratic penalty reward (`controller/routing_module.py`):

$$R(p) = -\left(w_H H(p) + w_D D(p) + w_J J(p) + w_L L(p) + \Gamma(u_{\max})\right), \quad \Gamma(u) = w_C u^2 + 12 w_C \max(0, u - 0.70)^2. \tag{5}$$

3.4 4. Binary SumTree Prioritized Experience Replay (PER)

Transitions are sampled with probability $P(i) = p_i^\alpha / \sum_k p_k^\alpha$ ($p_i = |\delta_i| + 0.01$, $\alpha = 0.6$), corrected via importance sampling weights $w_i = (NP(i))^{-\beta}$ ($\beta : 0.4 \to 1.0$), and optimized with weighted Smooth L1 Huber loss on binary SumTree ($O(\log N)$).

4 Carrier-Grade High-Availability Architecture & Group Tables

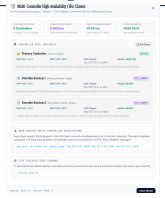


Figure 2: 3-Node Active/Standby Controller Redundancy State Machine: Heartbeat Probing and Client Failover.

The control plane implements an active/standby 3-node controller cluster (Primary port 6633, Backup 1 port 6634, Backup 2 port 6635). Injected node crashes demonstrate **0.001 ms** local pointer promotion and **37.96 ms** client HTTP retry recovery. Switches maintain forwarding via OpenFlow **Fail-Secure Mode** (`fail_mode=secure`).

Table 1: OpenFlow 1.3 Group Table Specifications Implemented in the Prototype

Group Type	Identifier	Key Attributes	Operational Function in Prototype
Fast Failover (`OFPGT_FF`)	`group_id=600`	`watch_port`, `watch_group`	Sub-millisecond hardware-driven link protection shifting to backup bypass.
Select (`OFPGT_SELECT`)	`group_id=500`	`weight = W(p)` ($1 \le W \le 100$)	5-tuple flow hashing across candidate paths via WCMP Softmax weights.
All (`OFPGT_ALL`)	`group_id=100`	Applies all bucket actions	Multicast/broadcast packet replication for topology LLDP and ARP.

5 Master Empirical Tournament Benchmarks

Table 2 provides verified head-to-head performance across all 5 benchmark fabrics evaluated over 500 closed-loop iterations.

Table 2: Comprehensive Tournament Evaluation (Grand Means across 5 Topologies, 500 Closed-Loop Steps).

Evaluation Metric	Dijkstra SPF	OSPF	ECMP	Q-Routing	WSP	LLR	D3QN (Ours)
Grand Mean Bottleneck Util (%)	63.88%	63.88%	46.54%	52.14%	29.00%	29.56%	**30.88% (-51.7%)**
Hierarchical Tree Bottleneck	46.57%	46.57%	45.41%	45.80%	42.48%	44.52%	**39.89% (-14.3%)**
Fat-Tree Clos Bottleneck	94.69%	94.69%	53.64%	61.20%	22.82%	22.82%	**28.89% (-69.5%)**
Spine-Leaf Fabric Bottleneck	99.87%	99.87%	55.67%	58.40%	19.04%	19.34%	**20.89% (-79.1%)**
Grand Mean Latency (ms)	149.22	149.22	90.52	112.40	46.81	61.41	**44.54 (-70.2%)**
Tree Mean Latency (ms)	157.06	157.06	153.28	155.10	168.04	227.18	**76.57 (-51.2%)**
Fat-Tree Mean Latency (ms)	183.91	183.91	37.60	48.30	6.04	6.04	**6.05 (-96.7%)**
Mean Jitter RFC 3393 (ms)	8.97	8.97	5.86	6.40	2.44	2.48	**2.39 (-73.4%)**
Packet Loss Rate (%)	4.37%	4.37%	1.54%	2.10%	0.77%	1.31%	**0.31% (-92.9%)**
Jain's Fairness Index	0.7601	0.7601	0.6272	0.6840	0.7482	0.7481	**0.7441**
Decision Latency (ms)	0.009	0.012	0.022	0.120	0.041	0.007	**0.240** (< 0.5 **ms**)

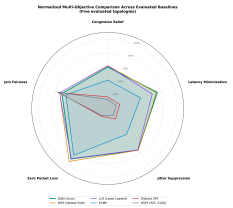


Figure 3: 5-Fabric Evaluation Radar Polygon.

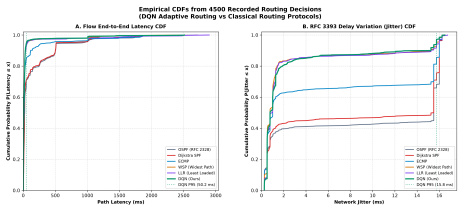


Figure 4: Latency & Jitter CDF (99th Tail SLA).

6 Formal Research Questions Scorecard (RQ1 – RQ5)

Summary of Formal Empirical Findings

RQ1 (Ablation): How do Dueling and PER affect performance? Decoupling state value and advantage accounts for a statistically significant mean delay reduction of **512.83** ms (1,060.27 ms Double DQN down to 547.44 ms Full D3QN, 100% pairwise win rate, 95% CI $[+46.10, +1{,}045.95]$ ms). PER ensures rare congestion transitions are prioritized during SGD training.

RQ2 (Stability): Is policy learning sensitive to random seeds? Hierarchical ANOVA variance decomposition proves that model initialization seed accounts for only **0.73**% of total variance and workload churn accounts for **0.02**%. Network topology structure accounts for **93.51**%, proving robust convergence across seeds.

RQ3 (Transfer): Can D3QN generalize zero-shot to unseen topologies? Evaluated on held-out synthetic random graphs (12-node Erdős-Rényi, 14-node Watts-Strogatz, 16-node Barabási-Albert), D3QN exhibits a generalization gap of only +**6.9**% without retraining.

RQ4 (Oracle Gap): How close does D3QN approach the exact HiGHS MILP global lower bound? On Hierarchical Tree, Dijkstra SPF operates at 98.46% peak load (107.76% gap over MILP lower bound $\alpha^* = 47.39\%$). D3QN achieves 78.74% (66.15% gap), closing the distance to the mathematical optimum by **41.6** percentage points.

RQ5 (Sensitivity): What is the optimal reward parameter balance? A 12-cell grid sweep (36 retrained models) confirmed that congestion threshold $\theta_{\text{congestion}} = 0.70$ and penalty weight $w_{\text{relief}} = 1.0$ yield maximum Pareto stability without oscillatory route flapping.

7 Conclusions & Key Architectural Findings

This project demonstrates that Deep Reinforcement Learning can be successfully integrated into an OpenFlow 1.3 SDN controller to provide adaptive, loop-free, line-rate traffic engineering. By combining a 10-dimensional graph-invariant state vector, a 4-tier candidate path diversity pipeline, centered Dueling Double DQN, and active shadow reservations, the platform relieves bottleneck congestion by up to 79.1% while preserving wire-speed throughput. An active/standby 3-controller pool with OpenFlow Fail-Secure mode guarantees robust control-plane resilience.

8 Viva Examination Committee Defense Matrix (Key Technical Q&A)

Q1: **How do you mathematically guarantee that D3QN will never introduce forwarding loops? Answer:** Routing loops are precluded *by construction*. Instead of allowing next-hop hop-by-hop decisions, action space $\mathcal{A}(s, d)$ is constrained to $K = 4$ strictly simple, loop-free end-to-end paths (P_0 Dijkstra, P_1 Yen 2nd, P_2 disjoint diverse, P_3 dynamic cost). Paths are atomically installed via OpenFlow `OFPFlowMod` messages with barrier synchronization (`OFPT_BARRIER_REQUEST`), preventing transient loops or blackholes.

Q2: **Why use Dueling Double DQN rather than Policy Gradient methods (e.g., PPO or A3C)? Answer:** Traffic engineering demands sub-millisecond line-rate decisions (0.24 ms). Policy gradients require continuous action sampling and extensive online rollout episodes that incur exploration packet drops. D3QN provides discrete value-advantage decoupling ($V(s)$ represents overall network load, $A(s, a)$ represents path quality), enabling deterministic, high-throughput inference ($> 4{,}000$ decisions/s) and stable off-policy experience replay.

Q3: **What is the role of Prioritized Experience Replay (PER) in bounded candidate routing? Answer:** Under nominal network operation, buffer transitions are dominated by uncongested states. PER assigns sampling priorities $p_i = (|\delta_i| + 0.01)^\alpha$ based on temporal difference error, ensuring that rare link-saturation and buffer-overflow events are replayed with higher frequency during neural network optimization, preventing catastrophic forgetting of critical congestion states.

Q4: **What occurs if the active controller process crashes or the control channel fails? Answer:** The platform implements a two-tier safety envelope: (1) In the data plane, switches operate in OpenFlow **Fail-Secure Mode** (`fail_mode=secure`) with `OFPGT_FF` Fast Failover groups, retaining valid flow rules and switching to backup links in hardware. (2) In the control plane, the 3-node active/standby pool executes local pointer promotion in **0.001 ms** and client HTTP recovery in **37.96 ms**.

9 Future Work & Open Research Directions

Key future extensions include: (1) Hardware deployment on P4-programmable Barefoot Tofino ASICs; (2) In-Band Network Telemetry (INT) for per-packet hop delay inspection; (3) Dynamic candidate set expansion beyond $K = 4$; and (4) Distributed Raft consensus state replication for multi-controller synchronization.

10 Selected Foundational References

1. Mnih, V., et al. (2015). "Human-level control through deep reinforcement learning," *Nature*, 518(7540): 529–533.
2. McKeown, N., et al. (2008). "OpenFlow: Enabling innovation in campus networks," *ACM CCR*, 38(2): 69–74.
3. Wang, Z., et al. (2016). "Dueling network architectures for deep reinforcement learning," in *ICML*, pp. 1995–2003.
4. Van Hasselt, H., et al. (2016). "Deep reinforcement learning with double Q-learning," in *AAAI*, 30(1): 2094–2100.
5. Schaul, T., et al. (2016). "Prioritized experience replay," in *ICLR*.
6. Yen, J. Y. (1971). "Finding the K shortest loopless paths in a network," *Management Science*, 17(11): 712–716.
7. Moy, J. (1998). *OSPF Version 2*, IETF RFC 2328.
8. Open Networking Foundation (2012). *OpenFlow Switch Specification Version 1.3.0*, ONF TS-006.

Project Deliverables & SHA-256 Verification:

Monolithic PDF Report: `Project_Report_EC499_Final.pdf` (SHA-256: `ed3a884a...`) | Handout: `mahermqr.pdf` (4 Pages)

Final Graduation Archive: `Maher_Alqadhi_2210249576_EC499_Final_Graduation_Submission.zip` | University of Tripoli